

“LINUX FILE SYSTEM EXPLORER USING SYSTEM CALLS”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

N. SAINATH REDDY

2520090115

M. SHIVA

2520090135

Under the Esteemed Guidance of

Dr. Y. Subbarayudu

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Academic Year 2026–2027

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “Linux File System Explorer Using System Calls” is a record of bonafide work of N. Sainath Reddy – 2520090115 and M. Shiva – 2520090135 submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The work presented in this report describes the design and implementation of a Linux-based file system explorer using C programming and Linux/POSIX system calls.

The project demonstrates direct interaction with the operating system for file, directory, navigation and metadata operations. The results embodied in this report are prepared for the academic project and have not been copied from any other department, university or institute.

N. SAINATH REDDY

2520090115

M. SHIVA

2520090135

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled “Linux File System Explorer Using System Calls” is a bonafide work done and submitted by N. Sainath Reddy – 2520090115 and M. Shiva – 2520090135, in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of Computer Science and Information Technology, K L (Deemed to be) University, during the academic year 2026–2027.

Signature of the Guide	Signature of the Course Coordinator	Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help, guidance and encouragement received from many people. We express our sincere thanks to everyone who supported us during the planning, development, testing and documentation of the Linux File System Explorer.

We express our deepest appreciation to our guide, Dr. Y. Subbarayudu, Department of Computer Science and Engineering, for his valuable guidance, suggestions and continuous encouragement throughout the project.

We are grateful to the Department of Computer Science and Information Technology and the faculty members for providing the academic environment and resources required to complete this project. We also thank the teaching and non-teaching staff members who assisted us directly or indirectly.

Finally, we sincerely thank our parents, friends and classmates for their support, cooperation and motivation during the completion of this work.

N. SAINATH REDDY

2520090115

M. SHIVA

2520090135

Abstract

Linux provides a powerful interface for managing files and directories through system calls. However, beginners often depend on graphical file managers without understanding how operating-system services actually perform file and directory operations. This project, “Linux File System Explorer Using System Calls,” develops a command-line file system explorer in C that demonstrates these operations directly through Linux/POSIX APIs.

The application provides functions for opening, reading, writing and closing files, moving through directories, listing directory contents, creating and removing directories, viewing file metadata and navigating between paths. The implementation is divided into modules such as file operations, directory operations, metadata, navigation and utility functions. This modular structure makes the program easier to understand, test and maintain.

The project is developed and tested in an Ubuntu/Linux environment. GCC is used for compilation and a Makefile automates the build process. Important system calls and library interfaces demonstrated by the project include `open()`, `read()`, `write()`, `close()`, `lseek()`, `stat()`, `opendir()`, `readdir()`, `mkdir()` and `rmdir()`. The project therefore connects programming concepts with practical operating-system behavior.

Overall, the system provides a simple and educational file explorer while demonstrating how a user program communicates with the Linux kernel through system-level interfaces.

Index

S. No.	Chapter Topics / Subtopics	Page No.
	Acknowledgement	i
	Abstract	ii
1	Introduction	1–4
1.1	Background of the Project	1
1.2	Problem Statement	1
1.3	Scope of the Project	2
1.4	Objectives of the Project	2
1.5	Importance of the Application	3
1.6	Target Users / Audience	4
2	OS Concepts and Background Study	5–8
2.1	Operating System Concepts Related to the Project	5
2.2	Linux File System and System Calls	6
2.3	Existing Methods / Approaches	7
2.4	Proposed Approach	8
3	System Requirements	9–11
3.1	Hardware Requirements	9
3.2	Software Requirements	9
3.3	Operating System Environment – Ubuntu / WSL	10
3.4	Development Tools and Compilers	11
4	Technology and Tools Used	12–15
4.1	C Programming	12
4.2	GCC Compiler and Make	12
4.3	Linux System Calls	13
4.4	Linux Commands and Utilities	14
4.5	Project File Structure	15
5	System Architecture and Design	16–20
5.1	Overall System Architecture	16
5.2	System Workflow	17
5.3	Flowchart / Data Flow	18
5.4	Module Description	19
5.5	Input and Output Design	20
6	Implementation	21–28

6.1	Module-wise Implementation	21
6.2	Algorithms / Pseudocode	23
6.3	Core C Implementation	24
6.4	Linux System Calls / APIs Used	25
6.5	Navigation and Metadata Implementation	27
6.6	Build Automation Using Makefile	28
7	Execution and Demonstration	29–33
7.1	Compilation in Ubuntu	29
7.2	Execution Commands	30
7.3	Sample Inputs and Outputs	31
7.4	Testing and Demonstration	32
7.5	Demonstration of OS Concepts	33
8	Conclusion	34–36
8.1	Summary of the Project	34
8.2	Objectives Achieved	35
8.3	Results and Learning Outcomes	35
8.4	Future Enhancements	36
9	References	37

CHAPTER - 1

INTRODUCTION

1.1 Background of the Project

A file system is one of the most important parts of an operating system. It provides a way to store, organize, access and manage files and directories on a storage device. Linux exposes many of these services to user programs through system calls and POSIX interfaces.

The Linux File System Explorer project is designed as a command-line application that demonstrates these operations in a simple and understandable way. Instead of depending on a graphical file manager, the program performs core operations through C code and operating-system interfaces.

1.2 Problem Statement

Many users can perform file operations through graphical applications but may not understand what happens between the application and the operating system. Students studying operating systems need practical examples showing how system calls are used for file access, directory handling, metadata retrieval and navigation.

The problem addressed by this project is to create a small, modular and easy-to-use explorer that demonstrates these low-level operations while remaining simple enough for academic learning.

1.3 Scope of the Project

- Browse and display the contents of directories.
- Navigate between directories and paths.
- Create and remove directories.
- Open, read, write and close files.
- Move the file offset using `lseek()`.
- Display file metadata using `stat()`.
- Provide a modular C implementation suitable for OS laboratory learning.
- Build and run the application using a Makefile in Ubuntu/WSL.

1.4 Objectives of the Project

- Understand how Linux system calls are used from a C program.
- Implement file and directory operations without relying on a high-level GUI.
- Demonstrate file metadata and directory traversal.
- Organize the project into separate source modules.
- Use GCC and Make to build the application efficiently.

1.5 Importance of the Application

The application is useful as an educational bridge between operating-system theory and programming practice. It allows students to observe how common file-management tasks can be implemented using operating-system interfaces. It also provides practical experience with error handling, file descriptors, paths, directory entries and metadata.

1.6 Target Users / Audience

- Students learning Operating Systems and System Programming.
- Students learning C programming on Linux.
- Beginners who want to understand Linux file operations.
- Faculty demonstrating system calls in laboratory sessions.

CHAPTER - 2

OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

An operating system manages hardware resources and provides services to application programs. File management is one of these services. When a C program requests a file operation, the request is handled through an operating-system interface.

Concept	Relation to the Project
System Calls	Provide controlled access to kernel services.
File Descriptor	Integer handle used to access an opened file.
Directory	Container used to organize files and subdirectories.
File Metadata	Information such as size, permissions and timestamps.
Path Navigation	Used to identify and move between locations.
Error Handling	Detects failures and reports useful information.

2.2 Linux File System and System Calls

The project demonstrates the difference between a user-level program and kernel-level services. The explorer runs as a user process, while Linux performs the requested low-level file-system operations. Functions such as `open()`, `read()`, `write()`, `close()`, `lseek()` and `stat()` are central to the implementation.

Directory traversal is handled using the POSIX directory interfaces `opendir()` and `readdir()`. Directory creation and removal are demonstrated using `mkdir()` and `rmdir()`.

2.3 Existing Methods / Approaches

A conventional desktop file manager provides a graphical interface for file and directory operations. While such applications are convenient, they hide most of the system-level details from the user. Standard shell commands such as `ls`, `cd`, `mkdir` and `rm` are also useful, but they do not by themselves show how a C application invokes the underlying interfaces.

2.4 Proposed Approach

The proposed approach is a modular command-line explorer written in C. Each major responsibility is separated into source files, making the implementation easier to read and maintain. The program communicates with Linux through file and directory APIs, reports errors, and provides an interactive way to perform common operations.

2.5 Advantages of the Proposed Approach

- Simple command-line interface.
- Direct demonstration of Linux/POSIX file APIs.
- Modular source-code organization.
- Easy compilation using Makefile.
- Suitable for Ubuntu and WSL environments.

- Useful for practical OS and system-programming demonstrations.

CHAPTER - 3

SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Minimum Requirement
Processor	Dual-core processor or better
RAM	4 GB or more
Storage	At least 1 GB free for tools and project
Display	Standard monitor
Keyboard	Standard keyboard

3.2 Software Requirements

Software	Purpose
Ubuntu Linux / Ubuntu on WSL	Development and execution environment
GCC	C compiler
GNU Make	Build automation
VS Code / Nano	Source-code editing
Bash Terminal	Running Linux commands

3.3 Operating System Environment – Ubuntu / WSL

The project can be developed directly on Ubuntu Linux or inside Ubuntu running through Windows Subsystem for Linux (WSL). WSL provides a Linux command-line environment where GCC, Make and POSIX interfaces can be used in a way that is close to a native Linux development setup.

```
sudo apt update
sudo apt install build-essential
cd ~/KLH_CSIT_2026-27_14_FileSystemExplorer/Project/LinuxFileSystemExplorer
```

3.4 Development Tools and Compilers

GCC is used to compile the C source files. GNU Make reads the project Makefile and builds only the required object files before linking them into the final executable. The project uses C11, warning flags and optimization settings.

```
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 -Iinclude -Isrc
```

3.5 Build Requirements

The Makefile expects the project to contain the src directory, include directory and the C source files listed below. The final executable is generated as build/explorer.

Source File	Main Responsibility
-------------	---------------------

main.c	Program entry point and main interaction flow.
file_operations.c	File-level operations.
directory_operations.c	Directory creation, removal and listing operations.
metadata.c	File and directory metadata.
navigation.c	Path and directory navigation.
utils.c	Common utility and support functions.

CHAPTER - 4

TECHNOLOGY AND TOOLS USED

4.1 C Programming

C is used because it provides low-level control and direct access to POSIX/Linux interfaces. It is well suited for demonstrating file descriptors, buffers, return values, error codes and system-level operations.

4.2 GCC Compiler and Make

GCC compiles each C source file into an object file. Make automates the complete build process and stores generated object files and the final executable inside the build directory.

4.3 Linux System Calls / APIs

API	Purpose in the Project
open()	Opens a file and returns a file descriptor.
read()	Reads bytes from an opened file.
write()	Writes bytes to a file.
close()	Closes an opened file descriptor.
lseek()	Changes the current file offset.
stat()	Obtains file metadata.
opendir()	Opens a directory stream.
readdir()	Reads directory entries.
mkdir()	Creates a directory.
rmdir()	Removes a directory.

4.4 Linux Commands and Utilities

- pwd – displays the current working directory.
- ls – displays directory contents.
- cd – changes the current directory.
- mkdir – creates a directory.
- rm/rmdir – removes files or directories according to the operation.
- gcc – compiles C source code.
- make – builds the project using the Makefile.

4.5 Project File Structure

```
LinuxFileSystemExplorer/  
├── Makefile  
├── include/  
│   └── project header files  
├── src/  
│   ├── main.c  
│   ├── file_operations.c  
│   ├── directory_operations.c  
│   ├── metadata.c  
│   ├── navigation.c  
│   └── utils.c  
└── build/  
    ├── *.o  
    └── explorer
```

The exact header-file names can vary with the project version, but the Makefile automatically includes all header files present under `include/` when compiling the source modules.

CHAPTER - 5

SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system follows a simple layered flow. The user interacts with the command-line explorer. The main program routes the requested operation to the appropriate module. The module then calls the required Linux/POSIX API. Linux performs the operation on the file system and returns a result or an error.

Layer	Description
User Layer	User enters commands or selects operations.
Application Layer	main.c controls the explorer flow.
Functional Modules	File, directory, metadata and navigation modules.
Linux/POSIX Interface	System calls and directory APIs.
File System	Actual files, directories and metadata.

5.2 System Workflow

The general workflow begins when the program starts and identifies the current working location. The user then chooses an operation. The corresponding module validates the input, invokes the Linux API and checks the returned value. The result is displayed to the user and the program continues until the user exits.

```
Start
↓
Initialize explorer
↓
Display current location / options
↓
Read user choice
↓
Select operation
↓
Call file/directory/navigation/metadata module
↓
Invoke Linux/POSIX API
↓
Check return value
↓
Display result or error
↓
Repeat until Exit
↓
End
```

5.3 Flowchart / Data Flow

Input flows from the terminal to the application modules. The modules convert the requested action into a Linux/POSIX operation. The operating system returns status information or data, which is then formatted and displayed by the explorer.

5.4 Module Description

Module	Description
main.c	Starts the application and manages the main user interaction.
file_operations.c	Handles file opening, reading, writing, closing and file-position operations.
directory_operations.c	Handles directory listing, creation and removal.
metadata.c	Reads and presents information about files and directories.
navigation.c	Supports movement between locations and path handling.
utils.c	Contains reusable helper and utility logic.

5.5 Input and Output Design

Inputs are primarily paths, file names, directory names and operation choices entered through the terminal. Outputs include directory listings, file contents, metadata, operation-success messages and error messages.

Input	Expected Output
Directory path	Directory contents or navigation result.
File path	File operation result / file content / metadata.
New directory name	Directory-created confirmation.
Existing directory name	Directory removal confirmation if valid.
Invalid path	Error message explaining the failure.

CHAPTER - 6

IMPLEMENTATION

6.1 Module-wise Implementation

The implementation is divided into independent source files. This avoids placing all logic inside main.c and makes individual operations easier to understand and test.

6.1.1 File Operations

File operations use file descriptors returned by `open()`. Data can be transferred using `read()` and `write()`, and the descriptor is released using `close()`. `lseek()` is used when the program needs to change the current position within a file.

```
fd = open(path, O_RDONLY);
if (fd == -1) {
    perror("open");
    return;
}
read(fd, buffer, size);
close(fd);
```

6.1.2 Directory Operations

Directory operations use `opendir()` to obtain a directory stream and `readdir()` to obtain directory entries. `mkdir()` is used to create directories and `rmdir()` is used to remove empty directories.

```
DIR *dir = opendir(path);
if (dir == NULL) {
    perror("opendir");
    return;
}

struct dirent *entry;
while ((entry = readdir(dir)) != NULL) {
    printf("%s\n", entry->d_name);
}
closedir(dir);
```

6.1.3 Metadata

The `stat()` API retrieves information about a file or directory. The returned structure can be used to obtain values such as file size, permissions, timestamps and file type.

```
struct stat st;
if (stat(path, &st) == -1) {
    perror("stat");
    return;
}
printf("Size: %lld\n", (long long)st.st_size);
```

6.1.4 Navigation

The navigation module manages the user's current location and path changes. It allows the explorer to work with different directories while keeping the command-line interaction understandable.

6.2 Algorithms / Pseudocode

6.2.1 Directory Listing Algorithm

```
1. Receive directory path.
2. Call opendir(path).
3. If the call fails, display the error.
4. Repeatedly call readdir().
5. Display each directory entry.
6. Call closedir().
7. Return to the main menu.
```

6.2.2 File Read Algorithm

```
1. Receive file path.
2. Call open() in read-only mode.
3. If open fails, display the error.
4. Read data using read().
5. Display the data.
6. Close the file using close().
7. Return to the main menu.
```

6.2.3 Metadata Algorithm

```
1. Receive path.
2. Call stat().
3. If stat fails, display the error.
4. Extract file type, size and other metadata.
5. Display the metadata.
6. Return to the main menu.
```

6.3 Core C Implementation

The implementation follows standard C11 compilation rules. Warning options are enabled so that common programming mistakes are easier to detect. The include paths allow the source files to access the project's header files.

```
CC = gcc
STD = -std=c11
WARN = -Wall -Wextra -Wpedantic
CFLAGS = $(STD) $(WARN) -O2 -Iinclude -Isrc
```

6.4 Linux System Calls / APIs Used

The core implementation depends on operating-system interfaces rather than implementing a separate file-system layer. Return values are checked after important operations so that failures such as invalid paths, permission problems or missing files can be reported.

6.5 Navigation and Metadata Implementation

Navigation combines path handling with directory operations. Metadata retrieval is separated into its own module so that file information can be displayed without duplicating the `stat()` logic throughout the application.

6.6 Build Automation Using Makefile

The Makefile defines the compiler, warning flags, source files, object files, build directory and executable target. It also provides `all`, `run`, `clean` and `help` targets.

```
make      # build the program
make run  # build if required and start the explorer
make clean # remove build outputs
make help # display available targets
```

The final executable is generated as `build/explorer`. The build directory is created automatically when required.

CHAPTER - 7

EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

Open the Ubuntu terminal and move to the project directory. Install the build tools if they are not already available.

```
sudo apt update
sudo apt install build-essential
cd ~/KLH_CSIT_2026-27_14_FileSystemExplorer/Project/LinuxFileSystemExplorer
make
```

When the build succeeds, the object files are placed inside build/ and the executable is generated as build/explorer.

7.2 Execution Commands

```
make run
# or
./build/explorer
```

7.3 Sample Inputs and Outputs

The following examples illustrate the type of interaction expected from the explorer. Exact menu wording may depend on the current source-code version.

```
Example:
Current Directory: /home/user/project

User selects directory listing
→ Displays files and folders in the current directory

User selects metadata
→ Displays information such as size and file type

User selects file read
→ Displays the contents of the selected file
```

7.4 Testing and Demonstration

Test Case	Input	Expected Result
Directory listing	Valid directory	Entries are displayed.
Invalid directory	Wrong path	Error is displayed.
File read	Existing readable file	File data is displayed.
Metadata	Existing path	Metadata is displayed.
Create directory	New directory name	Directory is created.
Remove directory	Empty directory	Directory is removed.
Build	make	Executable is generated.
Clean	make clean	Build directory is removed.

7.5 Demonstration of OS Concepts

- File descriptors are demonstrated during file operations.
- System-call/API return values are checked for success or failure.
- Directory streams demonstrate directory traversal.
- `stat()` demonstrates file metadata access.
- The Makefile demonstrates build automation in a Linux environment.
- Ubuntu/WSL demonstrates practical Linux system-programming execution.

CHAPTER - 8

CONCLUSION

8.1 Summary of the Project

The Linux File System Explorer Using System Calls project successfully demonstrates how a C application can perform common file-system tasks through Linux/POSIX interfaces. The modular design separates file operations, directory operations, metadata, navigation and utilities.

8.2 Objectives Achieved

- Implemented a Linux-based command-line file system explorer.
- Demonstrated important file and directory APIs.
- Implemented modular C source files.
- Used GCC and GNU Make for compilation and build automation.
- Tested the project in an Ubuntu/WSL environment.

8.3 Results and Learning Outcomes

The project provided practical understanding of system calls, file descriptors, directory streams, metadata structures, path handling and error checking. It also strengthened C programming, Linux command-line and build-system skills.

8.4 Future Enhancements

- Add recursive directory-size calculation.
- Add file copy, move and rename features.
- Add search by file name or extension.
- Add permission-management options.
- Improve the command-line interface with clearer menus and colors.
- Add optional logging of user operations.
- Provide a richer text-based interface while preserving the system-call focus.

CHAPTER - 9

REFERENCES

9.1 Books

- Operating System Concepts – Abraham Silberschatz, Peter B. Galvin and Greg Gagne.
- The Linux Programming Interface – Michael Kerrisk.
- The C Programming Language – Brian W. Kernighan and Dennis M. Ritchie.

9.2 Linux / GNU Documentation

- Linux/POSIX manual pages for `open()`, `read()`, `write()`, `close()`, `lseek()`, `stat()`, `opendir()`, `readdir()`, `mkdir()` and `rmdir()`.
- GCC documentation for C compilation and warning options.
- GNU Make documentation for build automation.

9.3 Online Resources

- Ubuntu documentation and Linux command-line resources.
- POSIX interface references used for system-programming study.

9.4 Project Source

Project source files and the Makefile are maintained in the Linux File System Explorer project directory. The report is structured to match the supplied academic project-documentation format, including the title page, declaration, certificate, acknowledgement, abstract, index and chapter-based technical documentation.